EdgeFHE 專題技術進度報告

EdgeFHE：KV260 邊緣端 CKKS 同態加密算術加速器

專題名稱：A Lightweight HW/SW Co-designed Accelerator for Client-Side CKKS Operations on FPGAs

平台：AMD Kria KV260 (`xck26-sfvc784-2LV-c`)

報告版本：V4 Candidate · 2026-10-09

目錄

1. 專題背景與目標
2. 系統架構設計
3. 各模組實作現況
4. 驗證成果與實驗數據
5. 與原始提案之對照分析
6. 已知限制
7. 後續工作規劃
8. 附錄：檔案結構與環境需求

1. 專題背景與目標

全同態加密（Fully Homomorphic Encryption, FHE）允許在密文上直接執行運算，無需解密即可取得正確結果。其中 CKKS 方案支援浮點近似運算，特別適用於隱私保護 AI 推論場景。然而，CKKS 加密過程需執行大量數論轉換（NTT）及模多項式運算，在邊緣端 ARM 處理器上耗時甚鉅。

本專題的核心目標為：在 AMD Kria KV260 SoC 平台上，透過**軟硬體協同設計**，將運算密集的 NTT 及多項式算術卸載至 FPGA 可程式邏輯（PL），由 ARM 處理器（PS）負責浮點編碼、隨機取樣與密文組裝，實現邊緣端的 CKKS client 加密加速。

1.1 固定參數

參數	值
多項式次數 N	4096
可用槽位數	2048
RNS 質數 q_0, q_1, q_2	998244353, 1004535809, 469762049
模數乘積 $Q = q_0 q_1 q_2$	4.711×10^{26}（$\log_2 Q \approx 88.61$）
編碼精度 Δ（V4）	2^{29}
負循環根 ψ	(350007156, 2062645, 62025685)
多項式環	$R_Q = \mathbb{Z}_Q[X]/(X^{4096}+1)$

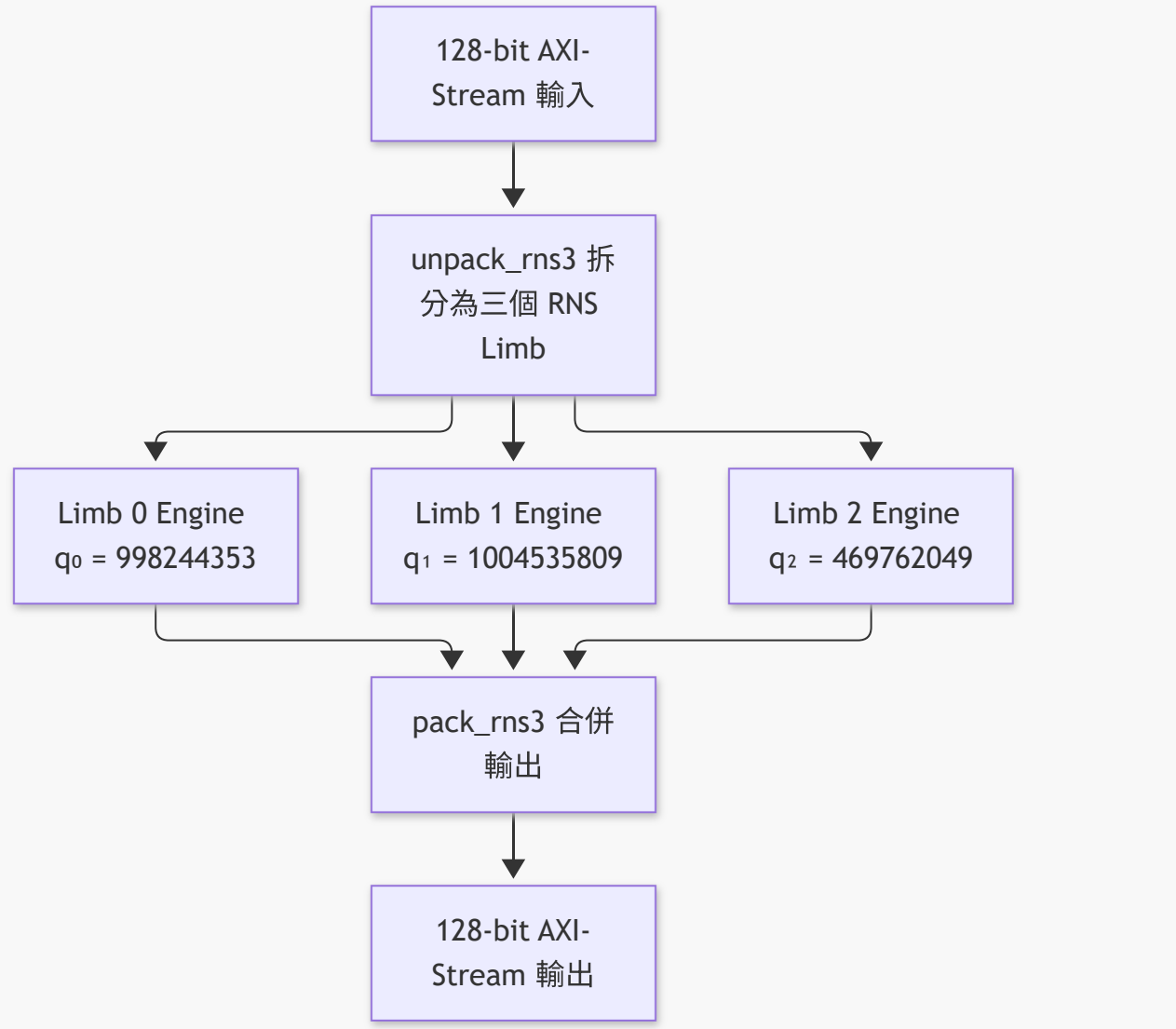
2. 系統架構設計

2.1 PS–PL 分工架構



- **PS（ARM Cortex-A53）**：負責 CKKS canonical encoding/decoding、OS 隨機數取樣、KeyGen/Encrypt/Decrypt 的密文組裝、CRT 重建及 sigmoid 分類決策。
- **PL（FPGA Fabric）**：負責 RNS3 NTT/INTT、逐點多項式乘法及向量加減運算。透過 128-bit AXI-Stream 與 PS 進行 DMA 資料搬移。

2.2 HLS 加速器頂層架構



三個 Limb Engine 透過 HLS `#pragma HLS DATAFLOW` 完全並行執行。每個 Engine 內含：

- **Four-Bank Butterfly Pipeline**：四蝶形單元並行，每批 8 讀 / 8 寫，迴圈啟動間隔 II=1

- **Montgomery Modular Reduction**：以常數乘法、低位截取及位移取代通用除法器，RTL 合成後確認無 runtime divider
- **Twiddle Factor Recurrence/Cache**：即時生成旋轉因子，減少 ROM 需求

2.3 FPGA 資源使用量（Routed Report）

資源	使用量	可用量	使用率
LUT	29,328	117,120	25.04%
FF	18,735	234,240	7.99%
DSP	753	1,248	60.34%
BRAM36 等效	62	144	43.06%
URAM	0	64	0%

時序：目標 10 ns (100 MHz)，WNS = +0.004 ns，WHS = +0.009 ns。

---

3. 各模組實作現況

3.1 FPGA 硬體加速器（已接受，V3 凍結）

HLS 頂層函式 `rns3_ntt_parallel_accel_n4096()` 支援六種操作：NTT（op0）、INTT（op1）、多項式乘法（op2）、二元加法（op3）、三元加法（op4）、取反加法（op5）。

Fixed-Bank 重構前後之 HLS 比較：

指標	重構前	重構後	變化
Butterfly 迴圈週期	2,054	515	−74.93%
迴圈 II	4	1	−75%
Pipeline 深度	11	5	−54.55%
最差延遲（cycles）	171,183	115,779	−32.37%

3.2 Linux DMA 驅動程式（已接受，V3 凍結）

- DMAEngine 框架，`dma_alloc_coherent()` 配合記憶體屏障
- 48-byte ioctl 請求結構（`_IOWR('E', 0x04, Request) = 0xc0304504`）
- Mutex 確保單筆 DMA 在途
- 32-bit DMA mask，TX 最大 196,608 bytes / RX 65,536 bytes

3.3 C++17 原生 CKKS 數學引擎（V4 新增）

V4 版本新增完整的 C++17 PS 數學流程，包含以下功能模組：

模組	實作內容
`encode()` / `decode()`	Canonical embedding：FFT + ζ-twist + ties-to-even 量化；decode 使用 signed 128-bit CRT 值
`sample_material()`	Linux `getrandom()` 系統呼叫，rejection sampling 避免模偏差，不使用 PRNG fallback
`keygen()`	$b = -a \cdot s + e$，公鑰可跨批次重用
`encrypt()`	四特徵各自加密：$c_0 = bu + e_0 + m$，$c_1 = au + e_1$；重用公鑰 NTT 結果
`decrypt()`	Centered CRT：$d = c_0 + c_1 \cdot s \pmod{Q}$，128-bit 中間值
`affine()`	同槽位權重走 $O(N)$ scalar 快徑；一般多項式權重走融合 NTT
`no_wrap_bound()`	整數界限檢查 $2U < Q$，防止 centered CRT 溢位
`sigmoid()` / `classify()`	穩定分支實作；不確定區間回傳 `None`，不強制判定

CPU NTT 使用 Shoup 預計算旋轉因子（$w' = \lfloor w \cdot 2^{32}/q \rfloor$），INTT 正規化與 untwist 融合。逐點乘積使用 Barrett reduction（$\mu = \lfloor 2^{64}/q \rfloor$），在 $q < 2^{30}$ 下一次修正即足。

3.4 兩機通訊協定（V4 新增）

項目	實作
傳輸安全	mTLS（TLS 1.2+），支援 client certificate 驗證
認證	Certificate DER SHA-256 pin + Bearer token
Port	從 HTTPS URL 動態取得，預設 443
封包	V4 profile ID 嚴格比對，request-response 綁定
服務端	有界 worker、replay ledger，僅接收密文，不記錄明文或私鑰
客戶端	stdin / CSV 動態輸入，1～2048 筆記錄，解密結果僅本機顯示

3.5 標準 OpenFHE 對照基準（V4 新增）

另建立獨立的 OpenFHE v1.5.1 完整 client baseline（`validation/openfhe_client/baseline.cpp`），使用 `HEStd_128_classic`、`UNIFORM_TERNARY`、N=4096、batch=2048、scale=29 配置。該基準使用不同的 moduli 與 sampler，僅作為獨立最佳化參考，不直接接入 PL。

---

4. 驗證成果與實驗數據

4.1 數學正確性驗證

驗證項目	規模	結果
Encoder Python / C++ 配對	24 cases，98,304 整數係數	Zero mismatch

驗證項目	規模	結果
三方 Dense NTT 驗證	1,000 組不同輸入，12,288,000 RNS 係數	Zero mismatch
完整 Client 流程	12 cases，1,867,776 整數 / RNS 比較	Zero mismatch
OpenFHE 低層向量	16 cases，Python / C / RTL 各 196,608 係數	全部通過
NTT / ALU 實板測試	25 cases，307,200 係數	全部通過
C++ Selftest (ASAN/UBSAN)	8,192 complex slots + 30,000 Barrett products	全部通過
動態 mTLS Loopback	2 ports、6 批（1/17/2048 records）	全部通過

Dense oracle 驗證方法：以 $B = 2^{80}$ 打包係數進行 Python 大整數 Kronecker 卷積，取得完整非循環卷積後做 $c[j] - c[j+N] \bmod q$，與 C++ NTT 結果及凍結 `uint64` 取餘 NTT 結果三方比對。此方法完全不依賴 FFT 或 NTT，排除共用 root/bit-reversal 的系統性錯誤。

4.2 實板穩定性測試（V3 凍結）

測試項目	結果
混合壓測	1,004 jobs，19 筆無 DMA 拒絕，兩 IRQ 各 +1,004
六小時穩定性	5,798,822 jobs，72 次正常 open/close
正常併發	2,000 jobs，同 process 四 threads
兩機 CKKS Affine	六例，84 DMA，1,032,192 係數，最大 slot error ≈ 1.156×10^{-5}

4.3 PS 數學效能比較（本機 x86_64 / WSL）

測試條件：WSL Ubuntu 24.04、GCC 13.3、30 配對、3 輪 warm-up、固定單核心 CPU affinity、交錯呼叫順序。數據僅含數學 hot path（含 FFI），排除 key setup、OS sampling、TLS 及網路。

運算階段	Python P50 (µs)	C++ P50 (µs)	比值
Encode（1 slot）	5,040	389	12.96
Encode（2048 slots）	5,024	875	5.74
KeyGen	39,671	1,232	32.21
Encrypt 四特徵	191,409	6,360	30.10
Scalar Affine	10,511	724	14.53
Decrypt / CRT	42,460	1,859	22.84
數學 Hot Path 合計	**284,313**	**12,565**	**22.63**

C++ 三 worker（RNS limb 平行）之 hot path P50 為 8,653 µs；滿 2048 筆 records 為 10,920 µs。

4.4 標準 OpenFHE CPU 基準

OpenMP 設定	Hot Path P50 (µs)	最大 Slot Error
1 thread	7,760	2.428×10^{-4}
3 threads	6,980	2.746×10^{-4}
4 threads	7,269	2.545×10^{-4}

> [!NOTE] OpenFHE 使用不同的 moduli、sampler 及 API，此數據僅作為獨立最佳化程度的參考基準，不構成安全等價或 FPGA 直接加速比的宣稱。

4.5 FPGA 單操作計時（V3 凍結，ARM Cortex-A53 單核心）

操作	樣本數	ARM P50 (µs)	FPGA Call P50 (µs)	ARM / FPGA
NTT	300	1,852	2,701	0.686
INTT	350	2,266	2,693	0.841
Multiply	150	6,591	5,026	1.311

FPGA call 包含 pack / ioctl / unpack 開銷。目前僅 multiply 操作呈現加速效果。

---

5. 與原始提案之關鍵差異與對照分析

5.1 架構設計的變更 (Architecture Shifts)

- **PRNG 雜訊生成由 PL 移至 PS**：原提案預期由 FPGA (PL) 硬體引擎生成雜訊多項式 e_0, e_1。但考量到硬體 PRNG 難以保證密碼學安全，V4 改由 **ARM (PS) 透過 Linux `getrandom()` 系統呼叫** 進行稀疏取樣，FPGA 則專注於 NTT 與模數乘法加速。
- **模數除法替代方案的定案**：提案預計使用 Barrett 或 Montgomery 來避開硬體除法。最終 V4 在 FPGA (PL) 端拍板使用 **Montgomery Reduction**（RTL 已無 runtime divider），而在 C++ (PS) 端則使用 **Shoup 旋轉因子與 Barrett Reduction**，針對軟硬體特性採用不同演算法。

5.2 尚未完成或延後的目標 (Pending Goals)

- **實體兩機 End-to-End 展示**：提案規劃在 PC 進行互動輸入並送往 FPGA 加密，再傳輸至 Cloud 進行 Logistic Regression 推論。V4 雖然完成了網路與運算程式，但**目前僅在「本機 (Loopback)」通過驗證**，實體跨機通訊與推論尚未完成板測驗收。
- **Twiddle Factor 即時生成**：提案期望以 On-the-fly 演算法即時生成旋轉因子以節省 BRAM。目前 HLS 仍採用 Cache/Recurrence 方式，具體的 ROM vs Cache 資源對照被列入 Phase 4 工作，尚未完成。
- **ARM 架構 (aarch64) 實測**：提案要求 C/C++ Driver 在 ARM Cortex 上運行。目前的 V4 C++17 程式碼與效能數據**皆於 x86_64 WSL 環境下測得**，尚未在 KV260 的 ARM 處理器上進行編譯與實測。

5.3 效能與預期的落差 (Performance Expectation)

- **FPGA 加速幅度**：提案預期硬體能帶來「顯著的速度提升」。但目前單次 FPGA 操作（含 DMA 開銷）中，NTT 甚至慢於 ARM（2.7ms vs 1.8ms），僅 Multiply 具備 **1.311 倍** 加速。完整的端到端 (E2E) 加速比仍待後續實測。反而是軟體端的 C++ 重構相較舊版 Python 帶來了高達 **22.63 倍** 的效能提升。

5.4 超出提案預期的增強 (Beyond Expectations)

- **完整的 C++17 CKKS 數學引擎**：提案僅要求實作 "C/C++ Encoder"，但 V4 實作了涵蓋 `KeyGen`、`Encrypt`、`Decrypt`、`CRT` 及 `Affine` 的完整 C++ 流程，並加入 128-bit 不溢位邊界檢查。
- **嚴謹的 mTLS 網路安全防護**：V4 實作了工業級的 mTLS（TLS 1.2+）雙向憑證認證與 Bearer Token 防重放攻擊，確保雲端伺服器對明文完全盲目，大幅提升系統安全性。
- **1000 組 Dense Oracle 測試**：V4 引入極其嚴格的數學驗證，執行了 1000 組不同輸入的比對（與 Python 卷積進行 Zero Mismatch 測試），並加入標準 `OpenFHE` 的 C++ 對照組，遠超出原提案的驗證規格。

5.5 提案需求達成矩陣

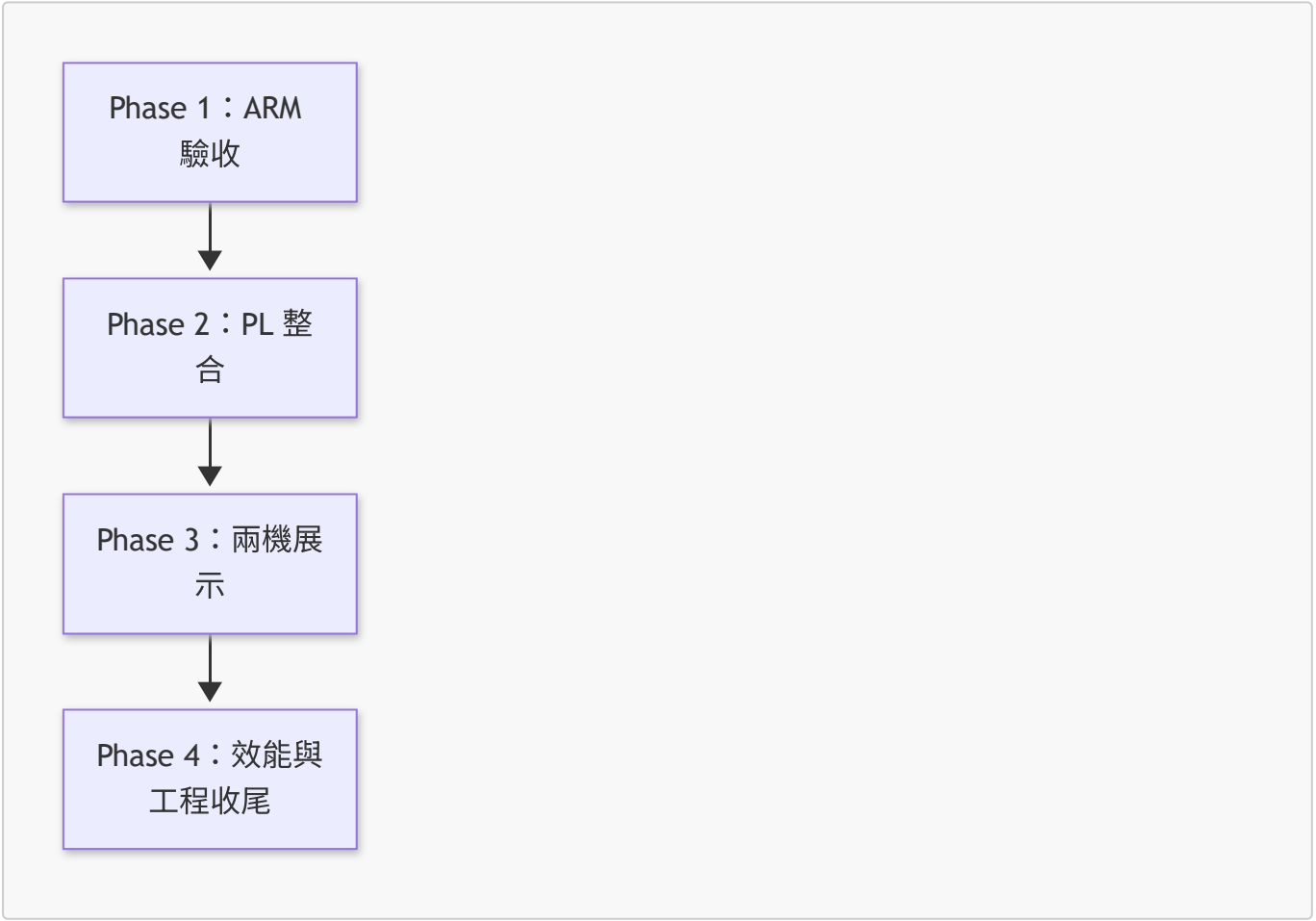
提案要求（頁碼）	實作現況	狀態
ARM 處理浮點編碼、FPGA 加速 NTT（第4頁）	PS 負責 encoding/decoding/sampling/組裝；PL 負責 NTT/INTT/乘法/ALU	✅ 已完成
C/C++ CKKS Encoder 與 ARM Driver（第1、4頁）	V4 新增完整 C++17 Encoder/KeyGen/Encrypt/Decrypt；Linux C Driver 已接受	✅ 已完成
HLS 流水線 NTT、Butterfly Unit（第4、6、16頁）	N=4096、三 RNS limbs × 四 lanes、butterfly II=1	✅ 已完成
Montgomery / Barrett 取代硬體除法（第16頁）	PL 使用 Montgomery，RTL 無通用除法器	✅ 已完成
AXI-Stream 與 ARM Driver 整合（第4、5頁）	128-bit AXIS + AXI-Lite、DMAEngine、48-byte ioctl	✅ 已完成
OpenFHE 驗證（第5頁）	16 組 OpenFHE 向量 + 標準 client baseline	✅ 已完成
排除 Edge Bootstrapping（第15頁）	未在 PL 實作 bootstrapping	✅ 符合設計
PC 互動輸入→Edge 加密→Cloud 推論→Edge 解密（第5頁）	V4 動態 mTLS + 任意輸入（本機驗證通過）	⚠️ 實體兩機待驗
相對純軟體有顯著加速（第4頁 MVP）	Multiply 操作 1.311×；C++ 較 Python 22.63×	⚠️ 部署 E2E 待量測
Logistic Regression 展示（第5頁）	Cloud 密文 affine + Edge sigmoid 分類	⚠️ 真實模型待驗收
PL PRNG 取樣（第1、5頁圖）	調整為 PS OS 隨機取樣	⚠️ 架構調整
敏感資料零洩漏目標（第5頁）	Ciphertext-only schema + mTLS	⚠️ 威脅模型待驗
On-the-fly Twiddle 減少 ROM（第5頁）	Recurrence/cache 已實作	⚠️ A/B 對照待做
G07 Physical Reset	Driver draft 有 lifecycle 模型	❌ 未接受
G08 Fault Recovery	Host counterexample + quarantine model	❌ 未接受

提案要求（頁碼）	實作現況	狀態
完整乾淨環境重建（G12）	共用建置文件與 CI 腳本	❌ 未完成

6. 已知限制

1. **V4 C++ 尚未於 KV260 ARM (aarch64) 上執行**：目前所有 V4 數學驗證與效能量測均在 x86_64 / WSL 環境完成，ARM 上的編譯與效能表現仍待實測。
2. **V4 native → PL 路徑未驗證**：V4 使用 Shoup/Barrett 的 CPU NTT 與凍結的 Montgomery HLS NTT 之間的 exact parity 尚未確認。
3. **部署 E2E 加速比未量測**：目前無法量化 ARM + FPGA 完整加密/解密流程相較於純 ARM 的加速效果。
4. **安全性未正式估算**：自訂稀疏取樣器的密碼安全等級未經分析，不宣稱符合 128-bit 安全標準。
5. **容錯處理不完整**：FPGA 跨模組 physical reset 與 DMA in-flight fault recovery 機制尚未完成板端驗收。
6. **本版為功能研究 MVP**：非正式工程發布版本，不應宣稱為安全產品或 100% 工程完成。

7. 後續工作規劃



Phase 1：ARM 驗收

- V4 C++17 aarch64 編譯與 CTest
- ARM Cortex-A53 CPU benchmark（30 配對量測）

- 確認 C++ 與 Python 在 ARM 平台上的效能差距

Phase 2：PL 整合

- V4 native C++ → PL exact parity 驗證
- 部署 hot path 設計：8 次 encrypt multiply + 1 次 final decrypt
- 量測 ARM + FPGA vs ARM-only 完整 E2E 加速比

Phase 3：兩機展示

- KV260 ↔ PC 實體 mTLS 連線
- 動態私密輸入 + 解密結果展示
- 公開模型與示範資料集驗收

Phase 4：效能與工程收尾

- G07/G08 容錯處理完善
- 安全 profile 估算
- Twiddle ROM vs Cache A/B 資源對照
- 溫度與功耗量測

---

8. 附錄：檔案結構與環境需求

8.1 專案目錄結構

```
EdgeFHE-KV260-GitHub/
├── hardware/
│   ├── hls/rns3_n4096/hls_source/src/    # HLS NTT 核心（凍結）
│   ├── ip/                               # 已接受 IP ZIP
│   └── vivado/                           # BD 重建腳本
├── firmware/ii1_n4096/                    # 凍結韌體
├── software/
│   ├── native_ckks/                      # V4 完整 C++17 CKKS
│   ├── ckks/                             # V3 Python 流程（凍結）
│   ├── driver/                           # V3 Linux Driver（已接受）
│   └── board_tests/                      # 板端測試程式
├── validation/
│   ├── vectors/                          # OpenFHE 驗證向量
│   └── openfhe_client/                   # 標準 OpenFHE baseline
├── evidence/                             # 接受矩陣與原始證據
├── docs/                                 # 技術文件（12 份）
├── tools/                                # 建置 / 檢查 / 驗證工具
└── release/                              # 版本 / 來源 / SHA256
```

8.2 開發環境需求

環境	需求
Python 檢查（Windows 可用）	Python 3.10+，僅標準庫

環境	需求
Native C++ 建置	Linux / WSL，GCC / Clang C++17，CMake ≥ 3.20
板端建置	Ubuntu 22.04 aarch64，matching kernel headers
HLS / Vivado 重建	Vitis HLS / Vivado 2022.1
OpenFHE baseline	OpenFHE v1.5.1 + OpenMP